

Report on Problem 1

December 4, 2025

1 Introduction

Lexical analysis, also known as scanning or tokenization, is the first phase of a compiler where the source code is broken down into a sequence of tokens. These tokens represent the basic elements of the programming language, such as keywords, identifiers, literals, operators, and punctuation symbols. The lexical analyzer (lexer) reads the input stream of characters and groups them into meaningful tokens while ignoring whitespace, comments, and other irrelevant parts.

Syntax analysis, or parsing, follows lexical analysis and involves checking whether the sequence of tokens forms a valid sentence according to the grammar rules of the language. It constructs a parse tree or abstract syntax tree (AST) that represents the hierarchical structure of the program. Tools like Bison are used to generate parsers based on context-free grammars.

In this report, we present a solution to a problem that demonstrates these analyses using Flex (for lexical analysis) and Bison (for syntax analysis) to process custom University and Department identifiers.

2 Problem Description

Consider the given statements:

1. RUET ##CSE_2018##
2. KUET ##EEE_2018\$#
3. BUET \$#ECE 2018##

a) Show a Flex file which can tokenize the given statements. b) Show a Bison file which can parse the given statements.

2.1 Lexical Analyzer (Flex File: 1.1)

```
%option noyywrap

%{
    #include <stdio.h>
    #include <string.h>
    #include <stdlib.h>
    #include "p.tab.h"
%}

uni (RUET|KUET|BUET)
dept1 (##CSE_)
dept2 (##EEE_)
dept3 (\$#ECE[ ])
s_char [#$]
```

```

digit [0-9]
year ({digit}{4})
ws [ \t\n]+
f_dept1 {dept1}{year}##
f_dept2 {dept2}{year}\$#
f_dept3 {dept3}{year}##

%%

{ws} {}
{uni} {printf("%s\n", yytext); return UNIV;}
{f_dept1} {printf("%s\n", yytext); return F_DEPT;}
{f_dept2} {printf("%s\n", yytext); return F_DEPT;}
{f_dept3} {printf("%s\n", yytext); return F_DEPT;}

%%

```

2.2 Parser (Bison File: p.y)

```

%{
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
void yyerror(const char *s);
int yylex();
extern int yylineno;
%}

%token UNIV F_DEPT
%start s

%%

s: s1 s
  | s1
  ;

s1: UNIV F_DEPT
   ;

%%

int main(void){
    printf("Starting Parsing:\n");
    yyparse();
    printf("Parsing completed.\n");
    return 0;
}

void yyerror(const char *s)
{
    fprintf(stderr, "Error: %s at line %d\n", s, yylineno);
    exit(1);
}

```

2.3 Input (Input.txt)

```
RUET  ##CSE_2018##  
KUET  ##EEE_2018$#  
BUET  $#ECE 2018##
```

2.4 Output (Output.txt)

```
Starting Parsing:  
RUET  
##CSE_2018##  
KUET  
##EEE_2018$#  
BUET  
$#ECE 2018##  
Parsing completed.
```

3 Conclusion

This report demonstrated the application of lexical and syntax analysis through a practical problem involving structured text data. Using Flex and Bison, we tokenized and parsed custom input statements containing University names and specific department codes. The solution highlights the importance of these compiler phases in ensuring correct structure in data streams. Future work could extend this to process more complex data formats or integrate semantic checks for validity of years and departments.